

Intel·ligència Artificial

Pràctica 3 — Planificació (PDDL)

Sistema de recomanació de viatges

Nivell bàsic + Extensions 1-4

Grau en Enginyeria Informàtica — FIB · UPC

Curs 2025/2026 · 2Q

Autors

Marcel Mula Tixier

Marc Planas Bosch

Jaime Prieto Salcedo

Índex

1. Introducció.....	4
1.1. Eina utilitzada.....	4
1.2. Metodologia de desenvolupament.....	4
2. Nivell bàsic.....	5
2.1. Modelat del domini.....	5
2.2. Modelat dels problemes.....	6
2.3. Jocs de prova i resultats.....	6
3. Extensió 1 — Dies.....	7
3.1. Canvis sobre el domini bàsic.....	7
3.2. Jocs de prova i resultats.....	7
3.3. Discussió: optimalitat.....	8
4. Extensió 2 — Interès.....	9
4.1. Canvis sobre el domini.....	9
4.2. Nota sobre la configuració del planificador.....	9
4.3. Jocs de prova i resultats.....	9
5. Extensió 3 — Preus.....	10
5.1. Canvis sobre el domini.....	10
5.2. Jocs de prova i resultats.....	10
6. Extensió 4 — Interès i preu ponderats.....	11
6.1. Domini i mètrica.....	11
6.2. Jocs de prova i efecte de la ponderació.....	11
7. Conclusions.....	12
Annex A. Codi PDDL.....	13
A.1. Nivell bàsic — domain.pddl.....	13
A.2. Nivell bàsic — problem1.pddl.....	13
A.3. Nivell bàsic — problem2.pddl.....	14
A.4. Extensió 1 — domain.pddl.....	15
A.5. Extensió 1 — problem1.pddl.....	15
A.6. Extensió 1 — problem2.pddl.....	16
A.7. Extensió 2 — domain.pddl.....	18
A.8. Extensió 2 — problem1.pddl.....	18
A.9. Extensió 2 — problem2.pddl.....	19
A.10. Extensió 3 — domain.pddl.....	21
A.11. Extensió 3 — problem1.pddl.....	22
A.12. Extensió 3 — problem2.pddl.....	23
A.13. Extensió 4 — domain.pddl.....	24
A.14. Extensió 4 — problem1.pddl.....	25
A.15. Extensió 4 — problem2.pddl.....	26
Annex B. Traces d'execució.....	28
B.1. Traces nivell bàsic.....	28
B.2. Traces Extensió 1.....	30
B.3. Traces Extensió 2.....	32
B.4. Traces Extensió 3.....	34

B.5. Traces Extensió 4.....	35
-----------------------------	----

1. Introducció

L'agència de viatges «al fin del mundo y más allá» vol un sistema que, a partir de la seva base de dades de ciutats, hotels i vols, i de certes restriccions de l'usuari, generi automàticament un pla de viatge. En aquesta pràctica modelem el problema en PDDL i el resollem amb un planificador en l'espai d'estats.

El planificador tria lliurement la ciutat d'inici i connecta diverses ciutats mitjançant un camí de vols sense repetir-ne cap. Els vols entre la ciutat origen del client i les ciutats inici/final del recorregut queden fora del problema, tal com indica l'enunciat.

1.1. Eina utilitzada

Hem fet servir Metric-FF v2.1, una variant de Fast-Forward que admet atributs numèrics (fluents) i comparacions que no són només d'igualtat — imprescindible perquè el problema compara comptadors (nombre de ciutats, dies) amb llindars mínims i màxims. El planificador s'executa per línia d'ordres:

```
.\ff.exe -o domain.pddl -f problem.pddl
```

Donat un objectiu numèric, Metric-FF mostra l'avís «numeric goal/precondition: turning cost-minimizing relaxed plans OFF». No és un error: només indica que desactiva una optimització interna de l'heurística. Per això cada problema declara una mètrica (:metric minimize ...) que orienta la cerca.

1.2. Metodologia de desenvolupament

Hem seguit un desenvolupament incremental basat en prototipatge ràpid, tal com recomana l'enunciat. Vam partir d'un primer prototip que cobreix el nivell bàsic, el vam validar amb jocs de prova, i a partir d'ell vam construir l'Extensió 1 reaprofitant predicats, operadors i jocs de prova i afegint només els elements nous (en aquest cas, la gestió dels dies). Cada prototip s'ha provat amb Metric-FF abans de passar al següent.

2. Nivell bàsic

Donat el conjunt de ciutats, els hotels de cada ciutat i els vols que les connecten, i un nombre mínim de ciutats a visitar, el sistema indica un hotel on allotjar-se i els vols per viatjar entre les ciutats, sense repetir-ne cap.

2.1. Modelat del domini

Tipus. Distingim dos tipus d'objectes: city (les ciutats) i hotel (els allotjaments, associats a una ciutat). Separar-los deixa el domini preparat per a les extensions de preus, on els hotels tindran atributs propis.

Predicats.

Predicat	Significat i justificació
(hotel-at ?h ?c)	L'hotel ?h és a la ciutat ?c. Relaciona hotels i ciutats; permet que el planificador esculli on allotjar-se.
(flight ?from ?to)	Hi ha un vol directe de ?from a ?to. Defineix el graf de connexions del problema.
(at ?c)	El viatger es troba ara a la ciutat ?c. Necessari per encadenar vols consecutius.
(visited ?c)	La ciutat ?c ja s'ha visitat. Garanteix que el camí sigui simple (no es repeteix cap ciutat).
(started)	El viatge ja ha començat. Distingeix la primera acció (triar inici) de la resta.
(booked ?h)	S'ha reservat l'hotel ?h. Fa que el pla mostri explícitament l'hotel escollit a cada ciutat.

Funcions (fluents numèrics).

Funció	Significat
(num-visited)	Comptador de ciutats visitades fins ara.
(min-cities)	Nombre mínim de ciutats que cal visitar (dada de l'usuari).

Aquests fluents són la raó per la qual cal *Metric-FF*: l'objectiu compara *num-visited* amb *min-cities* amb una desigualtat ($\geq$), no només amb igualtat.

Operadors.

Operador	Descripció i justificació
choose-start (?c ?h)	Tria la ciutat d'inici del recorregut i hi reserva hotel. L'enunciat exigeix que la ciutat inicial no estigui fixada: aquest operador permet que el planificador la triï lliurement. Marca <i>started</i> , <i>at</i> , <i>visited</i> , <i>booked</i> i incrementa <i>num-visited</i> .
travel (?from ?to ?h)	Viatja per un vol cap a una ciutat connectada i encara no visitada, i hi reserva hotel. Actualitza <i>at</i> , marca <i>visited</i> i <i>booked</i> , i

Operador	Descripció i justificació
	incrementa num-visited. La precondició (not (visited ?to)) impedeix repetir ciutats.

2.2. Modelat dels problemes

Un problema defineix els objectes (ciutats i hotels), l'estat inicial init (relacions hotel-at, vols flight, comptadors a zero i el mínim de ciutats) i el goal ($(\geq (\text{num-visited}) (\text{min-cities}))$). La mètrica (minimize (num-visited)) fa que, en absència d'altres criteris, el pla visiti exactament el mínim demanat (el camí més curt que compleix la restricció).

2.3. Jocs de prova i resultats

Hem dissenyat dos jocs de prova no trivials que proven aspectes diferents:

- Problema 1 — graf totalment connectat (4 ciutats, vols d'anada i tornada entre tots els parells), mínim 3 ciutats. Comprova que el planificador tria una ciutat d'inici i construeix un camí simple de 3 ciutats reservant hotel.
- Problema 2 — graf parcialment connex (6 ciutats en forma de cadena amb una branca, no totes les connexions existeixen), mínim 4 ciutats. Comprova que el planificador respecta les connexions reals i així i tot troba un recorregut vàlid.

Resultats obtinguts:

Problema	Pla trobat (cost)
Problema 1	choose-start bcn → travel par → travel rom. Cost 3 (3 ciutats, hotel a cadascuna). Correcte.
Problema 2	choose-start mad → travel bcn → travel par → travel ber. Cost 4. Respecta el graf parcial (no inventa vols inexistents). Correcte.

En tots dos casos el planificador tria sol la ciutat d'inici, no repeteix cap ciutat i assigna un hotel a cada ciutat visitada, complint el nivell bàsic. Les traces completes són a l'Annex B.

3. Extensió 1 — Dies

L'Extensió 1 amplia el nivell bàsic indicant el mínim i el màxim de dies que es pot estar a cada ciutat i el mínim de dies que ha de durar el recorregut. El pla resultant respecta aquestes restriccions i indica quants dies s'està a cada ciutat.

3.1. Canvis sobre el domini bàsic

Mantenim tipus, predicats i operadors del nivell bàsic i hi afegim el següent:

Noves funcions.

Funció	Significat
(total-days)	Dies totals acumulats del recorregut.
(days-in ?c)	Dies que es passa a la ciutat ?c.
(min-days-city)	Mínim de dies per ciutat.
(max-days-city)	Màxim de dies per ciutat.
(min-days-trip)	Mínim de dies del recorregut sencer.

Modelat dels dies.

En arribar a una ciutat (a choose-start i a travel) s'hi assignen automàticament els dies mínims (assign (days-in ?c) (min-days-city)) i s'acumulen a total-days. Així cada ciutat visitada compleix per construcció el mínim de dies.

Per allargar estades afegim un nou operador stay-extra-day (?c): suma un dia a la ciutat actual sempre que ($< (\text{days-in } ?c) (\text{max-days-city})$). D'aquesta manera mai se supera el màxim, i el planificador pot afegir dies fins a assolir el mínim del recorregut.

Objectiu i mètrica.

El goal exigeix alhora prou ciutats i prou dies: (and ($\geq (\text{num-visited}) (\text{min-cities})$) ($\geq (\text{total-days}) (\text{min-days-trip})$)). La mètrica (minimize (total-days)) busca l'estada més curta que compleix totes dues restriccions.

3.2. Jocs de prova i resultats

- Problema 1 — només 3 ciutats (graf complet), mínim 3 ciutats, dies per ciutat entre 2 i 4, recorregut mínim de 7 dies. Com que no hi ha cap quarta ciutat disponible, el planificador està obligat a allargar una estada: prova el mecanisme stay-extra-day en estat pur.
- Problema 2 — 6 ciutats en cadena, mínim 4 ciutats, dies per ciutat entre 1 i 3, recorregut mínim de 4 dies. Comprova la interacció entre dies i graf parcial.

Resultats obtinguts:

Problema	Pla trobat (cost = dies totals)
Problema 1	choose-start bcn → travel par → travel rom → stay-extra-day rom. 3 ciutats × 2 dies + 1 dia extra = 7 dies. Òptim.

Problema	Pla trobat (cost = dies totals)
Problema 2	choose-start mad → travel bcn → travel par → stay-extra-day par → travel ber. 4 ciutats + 1 dia extra = 5 dies. Vàlid.

3.3. Discussió: optimalitat

Durant les proves vam observar un cas il·lustratiu. En una versió inicial del Problema 1 amb 4 ciutats disponibles, Metric-FF retornava un pla vàlid però no òptim (visitava 4 ciutats, 8 dies, en lloc de 3 ciutats + 1 dia extra = 7). Això és coherent amb l'avís de l'enunciat: el planificador es guia per una estimació heurística independent del domini i no garanteix sempre la solució òptima. Per obtenir un joc de prova que demostrï netament el mecanisme de dies, vam restringir el Problema 1 a només 3 ciutats, forçant l'ús de stay-extra-day i obtenint l'òptim de 7 dies.

4. Extensió 2 — Interès

L'Extensió 2 amplia l'Extensió 1 assignant a cada ciutat un interès, representat com un enter (per exemple d'1 a 3, on 1 és el màxim interès). Volem un pla que maximitzi l'interès de les ciutats visitades; com que un interès més baix és millor, això equival a minimitzar la suma d'interessos.

4.1. Canvis sobre el domini

Es manté tot el model de l'Extensió 1 i s'hi afegeixen dues funcions:

Funció	Significat
(interest ?c)	Interès de la ciutat ?c (1 = màxim interès).
(total-interest)	Suma d'interessos de les ciutats visitades.

En arribar a una ciutat (a choose-start i a travel) s'acumula el seu interès a total-interest. La mètrica passa a ser (minimize (total-interest)): el planificador escull les ciutats de menor valor d'interès (és a dir, les més interessants) que compleixen la resta de restriccions.

4.2. Nota sobre la configuració del planificador

A partir d'aquesta extensió la mètrica és una magnitud numèrica a optimitzar (no només un comptador de passos). Vam observar que amb la configuració de cerca per defecte (EHC + A*epsilon, opció -s 5) Metric-FF retorna plans vàlids però sovint lluny de l'òptim en cost/interès. Per això executem aquestes extensions amb cerca per cost uniforme:

```
.\ff.exe -o domain.pddl -f problem.pddl -s 3 -w 0
```

L'opció -s 3 és A* ponderat amb minimització de cost i -w 0 fixa el pes de l'heurística a 0, de manera que la funció d'avaluació queda $f = g$ (cost acumulat real): és una cerca per cost uniforme que, en problemes de mida moderada com els nostres, troba la solució òptima. Aquesta decisió s'aplica també a les extensions 3 i 4.

4.3. Jocs de prova i resultats

- Problema 1 — graf complet de 5 ciutats amb interessos diferents (bcn=1, par=1, lon=2, rom=3, ams=3), mínim 3 ciutats. Comprova que el planificador tria les 3 ciutats de menor interès i evita les de valor 3.
- Problema 2 — graf parcial on les ciutats de millor interès no estan totes connectades entre si, mínim 4 ciutats. Obliga a un compromís entre interès i connectivitat.

Problema	Pla trobat (interès total)
Problema 1	par → lon → bcn. Interès 1+2+1 = 4 (evita rom i ams, de valor 3). Òptim.
Problema 2	ams → ber → par → bcn. Interès 1+1+3+1 = 6. Òptim (cap camí de 4 ciutats connex té menys interès).

En aquests problemes fixem els dies per ciutat ($\min = \max$) perquè el focus de l'extensió és l'interès i la traça quedi clara; el mecanisme de dies ja s'ha validat a l'Extensió 1.

5. Extensió 3 — Preus

L'Extensió 3 amplia l'Extensió 1 donant un preu als vols i als hotels. Volem minimitzar el preu total del pla, però indicant un preu mínim i un preu màxim totals que el pla ha de respectar.

5.1. Canvis sobre el domini

Aquesta extensió introdueix un canvi de modelat important: els vols passen a ser objectes (tipus flight). El motiu és que ara pot haver-hi diversos vols entre el mateix parell de ciutats amb preus diferents, i el planificador ha de poder triar quin agafa; un predicat binari (flight ?from ?to) no permetria distingir-los. Cada vol es descriu amb (flight-route ?f ?from ?to) i té un preu (flight-price ?f).

Els hotels tenen un preu per nit (hotel-price ?h). En arribar a una ciutat es paga $\text{hotel-price} \times \text{min-days-city}$, i l'operador stay-extra-day passa a pagar una nit addicional de l'hotel reservat. Així el preu depèn realment dels dies (model més fidel) i el planificador no afegeix nits supèrflues, perquè cada nit té cost.

Noves funcions: (total-price), (hotel-price ?h), (flight-price ?f), (min-price), (max-price).

L'objectiu hi afegeix la doble cota de preu: $(\geq (\text{total-price}) (\text{min-price}))$ i $(\leq (\text{total-price}) (\text{max-price}))$. La mètrica és (minimize (total-price)).

5.2. Jocs de prova i resultats

- Problema 1 — rang de preu ample (100..500). Hi ha dos vols bcn↔par (un de car, 100, i un de barat, 60) i dos hotels a París (80 i 50). En minimitzar, el planificador ha de triar el vol i l'hotel barats.
- Problema 2 — mateix mapa però amb preu mínim restrictiu (380..600). El pla més barat possible queda per sota del mínim, de manera que el planificador ha d'encarir el viatge fins a entrar dins el rang: demostra l'efecte del límit inferior.

Problema	Pla trobat (preu total)
Problema 1	rom → par (hotel barat) → bcn. Preu 310, el mínim possible dins el rang. Tria els vols i hotels barats.
Problema 2	lon → bcn → par (hotel barat). Preu 380, just el límit inferior: el planificador encareix el viatge (vol car lon→bcn) per superar el preu mínim sense passar-se.

6. Extensió 4 — Interès i preu ponderats

L'Extensió 4 uneix les extensions 2 i 3: el pla ha d'optimitzar alhora l'interès de les ciutats visitades i el preu total del recorregut. Com que interès (unitats) i preu (centenars) tenen escales molt diferents, la mètrica els combina amb ponderacions, i comprovem que pesos diferents donen plans diferents.

6.1. Domini i mètrica

El domini és la unió dels models anteriors: vols com a objectes amb preu, hotels amb preu per nit, dies mín/màx i interès per ciutat. La mètrica és una combinació lineal ponderada:

```
(:metric minimize (+ (* Wi (total-interest)) (* Wp (total-price))))
```

Wi i Wp són els pesos de l'interès i del preu. Atès que el preu pren valors molt més grans, amb pesos iguals el preu domina la decisió; per donar pes real a l'interès cal un Wi prou gran (de l'ordre de la relació d'escales).

6.2. Jocs de prova i efecte de la ponderació

Tots dos problemes comparteixen el mateix mapa, dissenyat amb una tensió clara: un grup de ciutats barates però d'interès dolent (bcn, lon, ams amb interès 3 i preus baixos) i un parell de ciutats molt interessants però cares (par, rom amb interès 1, hotels i vols cars). L'única diferència entre els dos problemes és el pes de l'interès a la mètrica:

- Problema 1 — ponderació orientada al preu: minimize (1·interès + 1·preu). Mana el preu.
- Problema 2 — ponderació orientada a l'interès: minimize (100·interès + 1·preu). El pes 100 compensa la diferència d'escales i fa que mani l'interès.

Problema	Pla trobat	Interès	Preu
P1 (pes preu)	ams → bcn → lon (ciutats barates)	9	220
P2 (pes interès)	ams → par → rom (ciutats interessants)	5	540

El resultat és el comportament desitjat: amb el mateix mapa, canviar només la ponderació canvia el pla. Quan mana el preu, el planificador es queda amb les ciutats barates encara que siguin poc interessants; quan mana l'interès, accepta pagar més per visitar les ciutats interessants. Això il·lustra com la mètrica ponderada permet equilibrar criteris amb unitats diferents.

7. Conclusions

Hem modelat en PDDL un sistema de recomanació de viatges i l'hem resolt amb Metric-FF, cobrint el nivell bàsic i les quatre extensions. El desenvolupament incremental ha estat clau: cada extensió reaprofitava el model anterior i hi afegia només els elements nous — els dies (Ext. 1) amb un operador i cinc funcions, l'interès (Ext. 2) amb dues funcions, els preus (Ext. 3) amb el pas dels vols a objectes i l'hotel per nit, i finalment la combinació ponderada (Ext. 4).

Les proves confirmen el comportament esperat en tots els nivells i ens han ajudat a entendre dues propietats del planificador. Primera, el seu caràcter heurístic: amb la cerca per defecte pot retornar plans vàlids però subòptims en cost. Segona, que escollint una cerca per cost uniforme (-s 3 -w 0) obtenim solucions òptimes en els nostres problemes. El cas de l'Extensió 4 mostra de manera especialment clara com la formulació de la mètrica (els pesos) determina la solució.

Annex A. Codi PDDL

A.1. Nivell bàsic — domain.pddl

```
(define (domain viatges-basic)

;; -----
;; NIVELL BÀSIC
;; Donat un conjunt de ciutats, els hotels de cada ciutat i els vols
;; que les connecten, i un nombre mínim de ciutats a visitar,
;; el planificador tria una ciutat d'inici, construeix un camí de vols
;; sense repetir ciutats i reserva un hotel a cada ciutat visitada.
;; -----

(:requirements :strips :typing :fluents)

(:types
  city hotel - object
)

(:predicates
  (hotel-at ?h - hotel ?c - city) ; l'hotel ?h es troba a la ciutat ?c
  (flight ?from - city ?to - city); existeix un vol directe de ?from a ?to
  (at ?c - city)                  ; el viatger es troba ara a la ciutat ?c
  (visited ?c - city)              ; la ciutat ?c ja ha estat visitada
  (started)                       ; el viatge ja ha començat
  (booked ?h - hotel)             ; s'ha reservat l'hotel ?h
)

(:functions
  (num-visited)  ; nombre de ciutats visitades fins ara
  (min-cities)   ; nombre mínim de ciutats que cal visitar
)

;; Tria la ciutat d'inici del recorregut i reserva-hi hotel.
;; El planificador decideix lliurement quina ciutat és l'origen.
(:action choose-start
  :parameters (?c - city ?h - hotel)
  :precondition (and (not (started))
                    (hotel-at ?h ?c))
  :effect (and (started)
              (at ?c)
              (visited ?c)
              (booked ?h)
              (increase (num-visited) 1))
)

;; Viatja d'una ciutat a una altra connectada per un vol,
;; sempre que la destinació no s'hagi visitat encara, i reserva-hi hotel.
(:action travel
  :parameters (?from - city ?to - city ?h - hotel)
  :precondition (and (started)
                    (at ?from)
                    (flight ?from ?to)
                    (not (visited ?to))
                    (hotel-at ?h ?to))
  :effect (and (not (at ?from))
              (at ?to)
              (visited ?to)
              (booked ?h)
              (increase (num-visited) 1))
)
)
```

A.2. Nivell bàsic — problem1.pddl

```
(define (problem viatge-basic-1)

;; -----
;; Joc de prova 1 (bàsic): graf petit i totalment connectat (anada/tornada
;; entre tots els parells). Es demana visitar com a mínim 3 ciutats.
;; Objectiu: comprovar que el planificador tria una ciutat d'inici i
;; construeix un camí de 3 ciutats sense repetir-ne cap, reservant hotel.
;; -----

(:domain viatges-basic)

(:objects
  bcn par rom lon - city
  h-bcn h-par h-rom h-lon - hotel
)
```

```

)

(:init
  ;; un hotel per ciutat
  (hotel-at h-bcn bcn)
  (hotel-at h-par par)
  (hotel-at h-rom rom)
  (hotel-at h-lon lon)

  ;; vols (ida i tornada) entre tots els parells
  (flight bcn par) (flight par bcn)
  (flight bcn rom) (flight rom bcn)
  (flight bcn lon) (flight lon bcn)
  (flight par rom) (flight rom par)
  (flight par lon) (flight lon par)
  (flight rom lon) (flight lon rom)

  (= (num-visited) 0)
  (= (min-cities) 3)
)

(:goal (>= (num-visited) (min-cities)))

;; sense altres criteris, minimitzem el nombre de ciutats: el pla visitarà
;; exactament el mínim demanat (camí més curt que compleix la restricció).
(:metric minimize (num-visited))
)

```

A.3. Nivell bàsic — problem2.pddl

```

(define (problem viatge-basic-2)

  ;; -----
  ;; Joc de prova 2 (bàsic): graf NO totalment connectat (en forma de
  ;; cadena/camí) i es demana visitar com a mínim 4 ciutats.
  ;; Objectiu: comprovar que el planificador respecta les connexions
  ;; reals (no totes les ciutats estan connectades entre si) i que tot i
  ;; així troba un recorregut vàlid que passa per 4 ciutats.
  ;; Connexions: mad - bcn - par - ber - ams (i tornada)
  ;;               \-- rom --/
  ;; -----

  (:domain viatges-basic)

  (:objects
    mad bcn par ber ams rom - city
    h-mad h-bcn h-par h-ber h-ams h-rom - hotel
  )

  (:init
    (hotel-at h-mad mad)
    (hotel-at h-bcn bcn)
    (hotel-at h-par par)
    (hotel-at h-ber ber)
    (hotel-at h-ams ams)
    (hotel-at h-rom rom)

    ;; graf parcialment connex (no totes les connexions existeixen)
    (flight mad bcn) (flight bcn mad)
    (flight bcn par) (flight par bcn)
    (flight par ber) (flight ber par)
    (flight ber ams) (flight ams ber)
    (flight par rom) (flight rom par)
    (flight rom ber) (flight ber rom)

    (= (num-visited) 0)
    (= (min-cities) 4)
  )

  (:goal (>= (num-visited) (min-cities)))

  ;; sense altres criteris, minimitzem el nombre de ciutats: el pla visitarà
  ;; exactament el mínim demanat (camí més curt que compleix la restricció).
  (:metric minimize (num-visited))
)

```

A.4. Extensió 1 — domain.pddl

```
(define (domain viatges-ext1)

;; -----
;; EXTENSIÓ 1 = Nivell bàsic + dies
;; S'indica el mínim i el màxim de dies que es pot estar a cada ciutat
;; i el mínim de dies que ha de durar el recorregut. El pla resultant
;; respecta aquestes restriccions i diu quants dies s'està a cada ciutat.
;;
;; Modelat dels dies: en arribar a una ciutat s'hi assignen els dies
;; mínims (min-days-city). L'acció stay-extra-day allarga l'estada d'un
;; dia, mai per sobre del màxim (max-days-city). El comptador total-days
;; acumula els dies del recorregut sencer.
;; -----

(:requirements :strips :typing :fluents)

(:types
  city hotel - object
)

(:predicates
  (hotel-at ?h - hotel ?c - city)
  (flight ?from - city ?to - city)
  (at ?c - city)
  (visited ?c - city)
  (started)
  (booked ?h - hotel)
)

(:functions
  (num-visited)      ; ciutats visitades
  (min-cities)       ; mínim de ciutats a visitar
  (total-days)       ; dies totals del recorregut
  (days-in ?c - city) ; dies que s'està a la ciutat ?c
  (min-days-city)    ; mínim de dies per ciutat
  (max-days-city)    ; màxim de dies per ciutat
  (min-days-trip)    ; mínim de dies del recorregut sencer
)

(:action choose-start
  :parameters (?c - city ?h - hotel)
  :precondition (and (not (started))
                     (hotel-at ?h ?c))
  :effect (and (started)
               (at ?c)
               (visited ?c)
               (booked ?h)
               (increase (num-visited) 1)
               (assign (days-in ?c) (min-days-city))
               (increase (total-days) (min-days-city)))
)

(:action travel
  :parameters (?from - city ?to - city ?h - hotel)
  :precondition (and (started)
                     (at ?from)
                     (flight ?from ?to)
                     (not (visited ?to))
                     (hotel-at ?h ?to))
  :effect (and (not (at ?from))
               (at ?to)
               (visited ?to)
               (booked ?h)
               (increase (num-visited) 1)
               (assign (days-in ?to) (min-days-city))
               (increase (total-days) (min-days-city)))
)

;; Allarga un dia l'estada a la ciutat actual, sense superar el màxim.
(:action stay-extra-day
  :parameters (?c - city)
  :precondition (and (at ?c)
                     (< (days-in ?c) (max-days-city)))
  :effect (and (increase (days-in ?c) 1)
               (increase (total-days) 1))
)
)
```

A.5. Extensió 1 — problem1.pddl

```

(define (problem viatge-ext1-1)

;; -----
;; Joc de prova 1 (ext.1): NOMÉS 3 ciutats (graf complet), mín 3 ciutats.
;; Dies per ciutat: entre 2 i 4. Mínim de dies del recorregut: 7.
;; Com que només hi ha 3 ciutats, ha de visitar-les totes (6 dies a
;; l'estada mínima) i NO pot afegir-ne cap més; per arribar als 7 dies
;; del recorregut està OBLIGAT a usar com a mínim un stay-extra-day.
;; -----

(:domain viatges-ext1)

(:objects
  bcn par rom - city
  h-bcn h-par h-rom - hotel
)

(:init
  (hotel-at h-bcn bcn)
  (hotel-at h-par par)
  (hotel-at h-rom rom)

  (flight bcn par) (flight par bcn)
  (flight bcn rom) (flight rom bcn)
  (flight par rom) (flight rom par)

  (= (num-visited) 0)
  (= (min-cities) 3)

  (= (total-days) 0)
  (= (min-days-city) 2)
  (= (max-days-city) 4)
  (= (min-days-trip) 7)
)

(:goal (and (>= (num-visited) (min-cities))
             (>= (total-days) (min-days-trip))))

;; minimitzem els dies totals: el pla farà l'estada més curta que
;; compleix alhora el mínim de ciutats i el mínim de dies del recorregut.
(:metric minimize (total-days))
)

```

A.6. Extensió 1 — problem2.pddl

```

(define (problem viatge-ext1-2)

;; -----
;; Joc de prova 2 (ext.1): graf parcialment connex (cadena amb branca),
;; mín 4 ciutats. Dies per ciutat entre 1 i 3, mínim de recorregut 4.
;; Objectiu: 4 ciutats a 1 dia ja arriben al mínim de 4 dies del
;; recorregut, de manera que NO cal cap stay-extra-day (estada mínima).
;; Es comprova el cas límit en què les restriccions ja es compleixen
;; amb l'estada mínima a cada ciutat.
;; -----

(:domain viatges-ext1)

(:objects
  mad bcn par ber ams rom - city
  h-mad h-bcn h-par h-ber h-ams h-rom - hotel
)

(:init
  (hotel-at h-mad mad)
  (hotel-at h-bcn bcn)
  (hotel-at h-par par)
  (hotel-at h-ber ber)
  (hotel-at h-ams ams)
  (hotel-at h-rom rom)

  (flight mad bcn) (flight bcn mad)
  (flight bcn par) (flight par bcn)
  (flight par ber) (flight ber par)
  (flight ber ams) (flight ams ber)
  (flight par rom) (flight rom par)
  (flight rom ber) (flight ber rom)

  (= (num-visited) 0)
  (= (min-cities) 4)

  (= (total-days) 0)
)

```

```
(= (min-days-city) 1)
(= (max-days-city) 3)
(= (min-days-trip) 4)
)

(:goal (and (>= (num-visited) (min-cities))
            (>= (total-days) (min-days-trip))))

(:metric minimize (total-days))
)
```

A.7. Extensió 2 — domain.pddl

```
(define (domain viatges-ext2)

;; -----
;; EXTENSIÓ 2 = Extensió 1 + interès
;; Cada ciutat té un interès enter (p. ex. 1..3, sent 1 el màxim interès).
;; Volem un pla que MINIMITZI la suma d'interessos de les ciutats
;; visitades (minimitzar el valor numèric = maximitzar l'interès real).
;; Es manté tot el model de l'Extensió 1 (dies mín/màx i dies del
;; recorregut) i s'afegeix l'acumulador d'interès.
;; -----

(:requirements :strips :typing :fluents)

(:types
  city hotel - object
)

(:predicates
  (hotel-at ?h - hotel ?c - city)
  (flight ?from - city ?to - city)
  (at ?c - city)
  (visited ?c - city)
  (started)
  (booked ?h - hotel)
)

(:functions
  (num-visited)
  (min-cities)
  (total-days)
  (days-in ?c - city)
  (min-days-city)
  (max-days-city)
  (min-days-trip)
  (interest ?c - city) ; interès de la ciutat ?c (1 = màxim interès)
  (total-interest) ; suma d'interessos de les ciutats visitades
)

(:action choose-start
  :parameters (?c - city ?h - hotel)
  :precondition (and (not (started))
                     (hotel-at ?h ?c))
  :effect (and (started)
               (at ?c)
               (visited ?c)
               (booked ?h)
               (increase (num-visited) 1)
               (assign (days-in ?c) (min-days-city))
               (increase (total-days) (min-days-city))
               (increase (total-interest) (interest ?c)))
)

(:action travel
  :parameters (?from - city ?to - city ?h - hotel)
  :precondition (and (started)
                     (at ?from)
                     (flight ?from ?to)
                     (not (visited ?to))
                     (hotel-at ?h ?to))
  :effect (and (not (at ?from))
               (at ?to)
               (visited ?to)
               (booked ?h)
               (increase (num-visited) 1)
               (assign (days-in ?to) (min-days-city))
               (increase (total-days) (min-days-city))
               (increase (total-interest) (interest ?to)))
)

(:action stay-extra-day
  :parameters (?c - city)
  :precondition (and (at ?c)
                     (< (days-in ?c) (max-days-city)))
  :effect (and (increase (days-in ?c) 1)
               (increase (total-days) 1))
)
)
```

A.8. Extensió 2 — problem1.pddl

```

(define (problem viatge-ext2-1)

;; -----
;; Joc de prova 1 (ext.2): graf complet de 5 ciutats amb interessos
;; diferents. Mín 3 ciutats. Dies poc restrictius (no interfereixen).
;; Interessos: bcn=1 par=1 lon=2 rom=3 ams=3
;; Objectiu: en minimitzar l'interès, el planificador hauria d'escollir
;; les 3 ciutats de menor valor (bcn, par, lon = 4) i evitar rom i ams.
;; -----

(:domain viatges-ext2)

(:objects
  bcn par lon rom ams - city
  h-bcn h-par h-lon h-rom h-ams - hotel
)

(:init
  (hotel-at h-bcn bcn) (hotel-at h-par par) (hotel-at h-lon lon)
  (hotel-at h-rom rom) (hotel-at h-ams ams)

  ;; graf complet (anada i tornada entre tots els parells)
  (flight bcn par) (flight par bcn)
  (flight bcn lon) (flight lon bcn)
  (flight bcn rom) (flight rom bcn)
  (flight bcn ams) (flight ams bcn)
  (flight par lon) (flight lon par)
  (flight par rom) (flight rom par)
  (flight par ams) (flight ams par)
  (flight lon rom) (flight rom lon)
  (flight lon ams) (flight ams lon)
  (flight rom ams) (flight ams rom)

  (= (interest bcn) 1)
  (= (interest par) 1)
  (= (interest lon) 2)
  (= (interest rom) 3)
  (= (interest ams) 3)

  (= (num-visited) 0)
  (= (min-cities) 3)

  ;; dies fixats (1 per ciutat): en l'ext.2 el focus és l'interès, així
  ;; que fixem els dies per ciutat (min = max) i la traça queda neta.
  (= (total-days) 0)
  (= (min-days-city) 1)
  (= (max-days-city) 1)
  (= (min-days-trip) 3)

  (= (total-interest) 0)
)

(:goal (and (>= (num-visited) (min-cities))
             (>= (total-days) (min-days-trip))))

(:metric minimize (total-interest))
)

```

A.9. Extensió 2 — problem2.pddl

```

(define (problem viatge-ext2-2)

;; -----
;; Joc de prova 2 (ext.2): graf PARCIAL on les ciutats de menor interès
;; NO estan totes connectades entre si, de manera que el planificador
;; ha d'arribar a un compromís entre interès i connectivitat.
;; Mín 4 ciutats.
;; Connexions: mad - bcn - par - ber - ams (cadena)
;;              \-- rom --/
;; Interessos: mad=2 bcn=1 par=3 ber=1 ams=1 rom=2
;; Les ciutats d'interès 1 (bcn, ber, ams) no formen un camí de 4 per
;; si soles; cal incloure-hi par (interès 3) o rom (interès 2).
;; -----

(:domain viatges-ext2)

(:objects
  mad bcn par ber ams rom - city
  h-mad h-bcn h-par h-ber h-ams h-rom - hotel
)

(:init

```

```

(hotel-at h-mad mad) (hotel-at h-bcn bcn) (hotel-at h-par par)
(hotel-at h-ber ber) (hotel-at h-ams ams) (hotel-at h-rom rom)

(flight mad bcn) (flight bcn mad)
(flight bcn par) (flight par bcn)
(flight par ber) (flight ber par)
(flight ber ams) (flight ams ber)
(flight par rom) (flight rom par)
(flight rom ber) (flight ber rom)

(= (interest mad) 2)
(= (interest bcn) 1)
(= (interest par) 3)
(= (interest ber) 1)
(= (interest ams) 1)
(= (interest rom) 2)

(= (num-visited) 0)
(= (min-cities) 4)

;; dies fixats (1 per ciutat): en l'ext.2 el focus és l'interès.
(= (total-days) 0)
(= (min-days-city) 1)
(= (max-days-city) 1)
(= (min-days-trip) 4)

(= (total-interest) 0)
)

(:goal (and (>= (num-visited) (min-cities))
            (>= (total-days) (min-days-trip))))

(:metric minimize (total-interest))
)

```

A.10. Extensió 3 — domain.pddl

```
(define (domain viatges-ext3)

;; -----
;; EXTENSIÓ 3 = Extensió 1 + preus
;; Els vols i els hotels tenen un preu. Volem minimitzar el preu total
;; del pla, respectant un preu mínim i un preu màxim total.
;;
;; Modelat:
;; - Els VOLS són objectes (tipus flight): pot haver-hi diversos vols
;;   entre el mateix parell de ciutats amb preus diferents i el
;;   planificador ha de triar.
;; - El preu de l'hotel és PER NIT. En arribar a una ciutat es paguen
;;   les nits mínimes (hotel-price * min-days-city) i cada stay-extra-day
;;   paga una nit addicional. Així els dies afecten el preu (realista) i
;;   el planificador no afegeix nits innecessàries.
;; -----

(:requirements :strips :typing :fluents)

(:types
  city hotel flight - object
)

(:predicates
  (hotel-at ?h - hotel ?c - city)
  (flight-route ?f - flight ?from - city ?to - city)
  (at ?c - city)
  (visited ?c - city)
  (started)
  (booked ?h - hotel)
  (flight-used ?f - flight)
)

(:functions
  (num-visited)
  (min-cities)
  (total-days)
  (days-in ?c - city)
  (min-days-city)
  (max-days-city)
  (min-days-trip)
  (total-price)
  (hotel-price ?h - hotel) ; preu per nit de l'hotel ?h
  (flight-price ?f - flight) ; preu del vol ?f
  (min-price)
  (max-price)
)

(:action choose-start
  :parameters (?c - city ?h - hotel)
  :precondition (and (not (started))
                    (hotel-at ?h ?c))
  :effect (and (started)
              (at ?c)
              (visited ?c)
              (booked ?h)
              (increase (num-visited) 1)
              (assign (days-in ?c) (min-days-city))
              (increase (total-days) (min-days-city))
              (increase (total-price) (* (hotel-price ?h) (min-days-city))))
)

(:action travel
  :parameters (?f - flight ?from - city ?to - city ?h - hotel)
  :precondition (and (started)
                    (at ?from)
                    (flight-route ?f ?from ?to)
                    (not (visited ?to))
                    (hotel-at ?h ?to))
  :effect (and (not (at ?from))
              (at ?to)
              (visited ?to)
              (booked ?h)
              (flight-used ?f)
              (increase (num-visited) 1)
              (assign (days-in ?to) (min-days-city))
              (increase (total-days) (min-days-city))
              (increase (total-price) (flight-price ?f))
              (increase (total-price) (* (hotel-price ?h) (min-days-city))))
)
```

```
;; Una nit més a la ciutat actual: paga una nit de l'hotel reservat.
(:action stay-extra-day
  :parameters (?c - city ?h - hotel)
  :precondition (and (at ?c)
                     (booked ?h)
                     (hotel-at ?h ?c)
                     (< (days-in ?c) (max-days-city)))
  :effect (and (increase (days-in ?c) 1)
               (increase (total-days) 1)
               (increase (total-price) (hotel-price ?h))))
)
```

A.11. Extensió 3 — problem1.pddl

```
(define (problem viatge-ext3-1)

  ;; -----
  ;; Joc de prova 1 (ext.3): minimitzar preu amb un rang ample (100..500).
  ;; Hi ha dos vols bcn->par (un car de 100 i un de barat de 60) i dos
  ;; hotels a par (un de 80 i un de 50). En minimitzar, el planificador
  ;; ha d'escollir el vol i l'hotel barats. Mín 3 ciutats.
  ;; -----

  (:domain viatges-ext3)

  (:objects
    bcn par rom lon - city
    h-bcn h-par1 h-par2 h-rom h-lon - hotel
    f-bp1 f-bp2 f-pb1 f-pb2 ; bcn->par (car/barat)
    f-pr f-rp f-r1 f-lr f-pl f-lp
    f-br f-rb f-bl f-lb - flight
  )

  (:init
    (hotel-at h-bcn bcn)
    (hotel-at h-par1 par) (hotel-at h-par2 par)
    (hotel-at h-rom rom)
    (hotel-at h-lon lon)

    (= (hotel-price h-bcn) 50)
    (= (hotel-price h-par1) 80)
    (= (hotel-price h-par2) 50)
    (= (hotel-price h-rom) 60)
    (= (hotel-price h-lon) 70)

    ;; vols i preus
    (flight-route f-bp1 bcn par) (= (flight-price f-bp1) 100)
    (flight-route f-bp2 bcn par) (= (flight-price f-bp2) 60)
    (flight-route f-pb1 par bcn) (= (flight-price f-pb1) 100)
    (flight-route f-pb2 par bcn) (= (flight-price f-pb2) 60)
    (flight-route f-pr par rom) (= (flight-price f-pr) 90)
    (flight-route f-rp rom par) (= (flight-price f-rp) 90)
    (flight-route f-r1 rom lon) (= (flight-price f-r1) 80)
    (flight-route f-lr lon rom) (= (flight-price f-lr) 80)
    (flight-route f-pl par lon) (= (flight-price f-pl) 110)
    (flight-route f-lp lon par) (= (flight-price f-lp) 110)
    (flight-route f-br bcn rom) (= (flight-price f-br) 120)
    (flight-route f-rb rom bcn) (= (flight-price f-rb) 120)
    (flight-route f-bl bcn lon) (= (flight-price f-bl) 150)
    (flight-route f-lb lon bcn) (= (flight-price f-lb) 150)

    (= (num-visited) 0)
    (= (min-cities) 3)

    (= (total-days) 0)
    (= (min-days-city) 1)
    (= (max-days-city) 2)
    (= (min-days-trip) 3)

    (= (total-price) 0)
    (= (min-price) 100)
    (= (max-price) 500)
  )

  (:goal (and (>= (num-visited) (min-cities))
              (>= (total-days) (min-days-trip))
              (>= (total-price) (min-price))
              (<= (total-price) (max-price))))

  (:metric minimize (total-price))
)
```

)

A.12. Extensió 3 — problem2.pddl

```
(define (problem viatges-ext3-2)

;; -----
;; Joc de prova 2 (ext.3): el PREU MÍNIM és restrictiu.
;; Mateix mapa que el problema 1 però amb rang 380..600. El pla més
;; barat possible (uns 310) queda PER SOTA del mínim, de manera que el
;; planificador està obligat a encarir el viatge (vol/hotel més cars,
;; o més ciutats) fins a entrar dins el rang [380, 600].
;; Demostra l'efecte del límit inferior de preu.
;; -----

(:domain viatges-ext3)

(:objects
  bcn par rom lon - city
  h-bcn h-par1 h-par2 h-rom h-lon - hotel
  f-bp1 f-bp2 f-pb1 f-pb2
  f-pr f-rp f-rl f-lr f-pl f-lp
  f-br f-rb f-bl f-lb - flight
)

(:init
  (hotel-at h-bcn bcn)
  (hotel-at h-par1 par) (hotel-at h-par2 par)
  (hotel-at h-rom rom)
  (hotel-at h-lon lon)

  (= (hotel-price h-bcn) 50)
  (= (hotel-price h-par1) 80)
  (= (hotel-price h-par2) 50)
  (= (hotel-price h-rom) 60)
  (= (hotel-price h-lon) 70)

  (flight-route f-bp1 bcn par) (= (flight-price f-bp1) 100)
  (flight-route f-bp2 bcn par) (= (flight-price f-bp2) 60)
  (flight-route f-pb1 par bcn) (= (flight-price f-pb1) 100)
  (flight-route f-pb2 par bcn) (= (flight-price f-pb2) 60)
  (flight-route f-pr par rom) (= (flight-price f-pr) 90)
  (flight-route f-rp rom par) (= (flight-price f-rp) 90)
  (flight-route f-rl rom lon) (= (flight-price f-rl) 80)
  (flight-route f-lr lon rom) (= (flight-price f-lr) 80)
  (flight-route f-pl par lon) (= (flight-price f-pl) 110)
  (flight-route f-lp lon par) (= (flight-price f-lp) 110)
  (flight-route f-br bcn rom) (= (flight-price f-br) 120)
  (flight-route f-rb rom bcn) (= (flight-price f-rb) 120)
  (flight-route f-bl bcn lon) (= (flight-price f-bl) 150)
  (flight-route f-lb lon bcn) (= (flight-price f-lb) 150)

  (= (num-visited) 0)
  (= (min-cities) 3)

  (= (total-days) 0)
  (= (min-days-city) 1)
  (= (max-days-city) 2)
  (= (min-days-trip) 3)

  (= (total-price) 0)
  (= (min-price) 380)
  (= (max-price) 600)
)

(:goal (and (>= (num-visited) (min-cities))
             (>= (total-days) (min-days-trip))
             (>= (total-price) (min-price))
             (<= (total-price) (max-price))))

(:metric minimize (total-price))
)
```

A.13. Extensió 4 — domain.pddl

```
(define (domain viatges-ext4)

;; -----
;; EXTENSIÓ 4 = Extensió 2 + Extensió 3
;; El pla optimitza alhora l'interès de les ciutats visitades i el preu
;; total del recorregut. Com que interès i preu tenen unitats molt
;; diferents, la mètrica del problema combina tots dos amb PONDERACIONS
;; (pesos), i comprovem que pesos diferents donen plans diferents.
;;
;; Reuneix tot el model anterior: vols com a objectes amb preu, hotels
;; amb preu per nit, dies mín/màx, i interès per ciutat.
;; -----

(:requirements :strips :typing :fluents)

(:types
  city hotel flight - object
)

(:predicates
  (hotel-at ?h - hotel ?c - city)
  (flight-route ?f - flight ?from - city ?to - city)
  (at ?c - city)
  (visited ?c - city)
  (started)
  (booked ?h - hotel)
  (flight-used ?f - flight)
)

(:functions
  (num-visited)
  (min-cities)
  (total-days)
  (days-in ?c - city)
  (min-days-city)
  (max-days-city)
  (min-days-trip)
  (total-price)
  (hotel-price ?h - hotel)
  (flight-price ?f - flight)
  (min-price)
  (max-price)
  (interest ?c - city)
  (total-interest)
)

(:action choose-start
  :parameters (?c - city ?h - hotel)
  :precondition (and (not (started))
                    (hotel-at ?h ?c))
  :effect (and (started)
              (at ?c)
              (visited ?c)
              (booked ?h)
              (increase (num-visited) 1)
              (assign (days-in ?c) (min-days-city))
              (increase (total-days) (min-days-city))
              (increase (total-price) (* (hotel-price ?h) (min-days-city)))
              (increase (total-interest) (interest ?c)))
)

(:action travel
  :parameters (?f - flight ?from - city ?to - city ?h - hotel)
  :precondition (and (started)
                    (at ?from)
                    (flight-route ?f ?from ?to)
                    (not (visited ?to))
                    (hotel-at ?h ?to))
  :effect (and (not (at ?from))
              (at ?to)
              (visited ?to)
              (booked ?h)
              (flight-used ?f)
              (increase (num-visited) 1)
              (assign (days-in ?to) (min-days-city))
              (increase (total-days) (min-days-city))
              (increase (total-price) (flight-price ?f))
              (increase (total-price) (* (hotel-price ?h) (min-days-city)))
              (increase (total-interest) (interest ?to)))
)
```

```

(:action stay-extra-day
  :parameters (?c - city ?h - hotel)
  :precondition (and (at ?c)
                     (booked ?h)
                     (hotel-at ?h ?c)
                     (< (days-in ?c) (max-days-city)))
  :effect (and (increase (days-in ?c) 1)
              (increase (total-days) 1)
              (increase (total-price) (hotel-price ?h))))
)
)

```

A.14. Extensió 4 — problem1.pddl

```

(define (problem viatge-ext4-1)

;; -----
;; Joc de prova 1 (ext.4): PONDERACIÓ ORIENTADA AL PREU.
;; Mètrica = 1*interès + 1*preu → com que els preus (centenars) són molt
;; més grans que els interessos (unitats), de fet mana el preu.
;;
;; Mapa (compartit amb el problema 2): hi ha un "triangle avorrit" de
;; ciutats barates però d'interès dolent (bcn, lon, ams: interès 3) i un
;; parell de ciutats "interessants" (par, rom: interès 1) que són cares
;; d'hotel i de vol. Min 3 ciutats.
;;
;; Resultat esperat: el pla barat → visita el triangle avorrit
;; (interès alt = dolent, però preu baix).
;; -----

(:domain viatges-ext4)

(:objects
  bcn lon ams par rom - city
  h-bcn h-lon h-ams h-par h-rom - hotel
  f-bl f-lb f-la f-al f-ba f-ab ; triangle barat (avorrit)
  f-pr f-rp ; enllaç interessant barat
  f-bp f-pb f-lp f-pl f-ar f-ra f-ap f-pa ; avorrit<->interessant (cars)
  f-br f-rb - flight
)

(:init
  (hotel-at h-bcn bcn) (hotel-at h-lon lon) (hotel-at h-ams ams)
  (hotel-at h-par par) (hotel-at h-rom rom)

  ;; hotels (preu per nit): barats els avorrits, cars els interessants
  (= (hotel-price h-bcn) 40) (= (hotel-price h-lon) 40) (= (hotel-price h-ams) 40)
  (= (hotel-price h-par) 120) (= (hotel-price h-rom) 120)

  ;; interès (1 = màxim interès): avorrits=3, interessants=1
  (= (interest bcn) 3) (= (interest lon) 3) (= (interest ams) 3)
  (= (interest par) 1) (= (interest rom) 1)

  ;; triangle barat (avorrit)
  (flight-route f-bl bcn lon) (= (flight-price f-bl) 50)
  (flight-route f-lb lon bcn) (= (flight-price f-lb) 50)
  (flight-route f-la lon ams) (= (flight-price f-la) 50)
  (flight-route f-al ams lon) (= (flight-price f-al) 50)
  (flight-route f-ba bcn ams) (= (flight-price f-ba) 50)
  (flight-route f-ab ams bcn) (= (flight-price f-ab) 50)

  ;; enllaç entre ciutats interessants (barat)
  (flight-route f-pr par rom) (= (flight-price f-pr) 60)
  (flight-route f-rp rom par) (= (flight-price f-rp) 60)

  ;; connexions avorrit <-> interessant (cares)
  (flight-route f-bp bcn par) (= (flight-price f-bp) 200)
  (flight-route f-pb par bcn) (= (flight-price f-pb) 200)
  (flight-route f-lp lon par) (= (flight-price f-lp) 200)
  (flight-route f-pl par lon) (= (flight-price f-pl) 200)
  (flight-route f-ar ams rom) (= (flight-price f-ar) 200)
  (flight-route f-ra rom ams) (= (flight-price f-ra) 200)
  (flight-route f-ap ams par) (= (flight-price f-ap) 200)
  (flight-route f-pa par ams) (= (flight-price f-pa) 200)
  (flight-route f-br bcn rom) (= (flight-price f-br) 200)
  (flight-route f-rb rom bcn) (= (flight-price f-rb) 200)

  (= (num-visited) 0)
  (= (min-cities) 3)

  (= (total-days) 0)
)

```

```

(= (min-days-city) 1)
(= (max-days-city) 1)
(= (min-days-trip) 3)

(= (total-price) 0)
(= (min-price) 1)
(= (max-price) 2000)

(= (total-interest) 0)
)

(:goal (and (>= (num-visited) (min-cities))
            (>= (total-days) (min-days-trip))
            (>= (total-price) (min-price))
            (<= (total-price) (max-price))))

;; ponderació orientada al preu (interès amb pes 1)
(:metric minimize (+ (* 1 (total-interest)) (* 1 (total-price))))
)

```

A.15. Extensió 4 — problem2.pddl

```

(define (problem viatge-ext4-2)

;; -----
;; Joc de prova 2 (ext.4): PONDERACIÓ ORIENTADA A L'INTERÈS.
;; MATEIX MAPA que el problema 1, però mètrica = 100*interès + 1*preu.
;; Ara el pes de l'interès és prou gran per compensar la diferència
;; d'unitats, de manera que el planificador prioritza visitar ciutats
;; de bon interès (par, rom) encara que siguin més cares.
;;
;; Resultat esperat: un pla DIFERENT del problema 1 – inclou les ciutats
;; interessants (interès baix = bo) tot i pagar més.
;; Comparant els dos problemes es veu l'efecte de la ponderació.
;; -----

(:domain viatges-ext4)

(:objects
  bcn lon ams par rom - city
  h-bcn h-lon h-ams h-par h-rom - hotel
  f-bl f-lb f-la f-al f-ba f-ab
  f-pr f-rp
  f-bp f-pb f-lp f-pl f-ar f-ra f-ap f-pa
  f-br f-rb - flight
)

(:init
  (hotel-at h-bcn bcn) (hotel-at h-lon lon) (hotel-at h-ams ams)
  (hotel-at h-par par) (hotel-at h-rom rom)

  (= (hotel-price h-bcn) 40) (= (hotel-price h-lon) 40) (= (hotel-price h-ams) 40)
  (= (hotel-price h-par) 120) (= (hotel-price h-rom) 120)

  (= (interest bcn) 3) (= (interest lon) 3) (= (interest ams) 3)
  (= (interest par) 1) (= (interest rom) 1)

  (flight-route f-bl bcn lon) (= (flight-price f-bl) 50)
  (flight-route f-lb lon bcn) (= (flight-price f-lb) 50)
  (flight-route f-la lon ams) (= (flight-price f-la) 50)
  (flight-route f-al ams lon) (= (flight-price f-al) 50)
  (flight-route f-ba bcn ams) (= (flight-price f-ba) 50)
  (flight-route f-ab ams bcn) (= (flight-price f-ab) 50)

  (flight-route f-pr par rom) (= (flight-price f-pr) 60)
  (flight-route f-rp rom par) (= (flight-price f-rp) 60)

  (flight-route f-bp bcn par) (= (flight-price f-bp) 200)
  (flight-route f-pb par bcn) (= (flight-price f-pb) 200)
  (flight-route f-lp lon par) (= (flight-price f-lp) 200)
  (flight-route f-pl par lon) (= (flight-price f-pl) 200)
  (flight-route f-ar ams rom) (= (flight-price f-ar) 200)
  (flight-route f-ra rom ams) (= (flight-price f-ra) 200)
  (flight-route f-ap ams par) (= (flight-price f-ap) 200)
  (flight-route f-pa par ams) (= (flight-price f-pa) 200)
  (flight-route f-br bcn rom) (= (flight-price f-br) 200)
  (flight-route f-rb rom bcn) (= (flight-price f-rb) 200)

  (= (num-visited) 0)
  (= (min-cities) 3)

  (= (total-days) 0)
)

```

```

    (= (min-days-city) 1)
    (= (max-days-city) 1)
    (= (min-days-trip) 3)

    (= (total-price) 0)
    (= (min-price) 1)
    (= (max-price) 2000)

    (= (total-interest) 0)
  )

  (:goal (and (>= (num-visited) (min-cities))
              (>= (total-days) (min-days-trip))
              (>= (total-price) (min-price))
              (<= (total-price) (max-price))))

  ;; ponderació orientada a l'interès (interès amb pes 100)
  (:metric minimize (+ (* 100 (total-interest)) (* 1 (total-price))))
)

```

Annex B. Traces d'execució

B.1. Traces nivell bàsic

```
#####
# TRACES NIVELL BASIC (metric-ff v2.1)
#####

===== problem1 (graf complet, min 3) =====

ff: parsing domain file
domain 'VIATGES-BASIC' defined
... done.
ff: parsing problem file
problem 'VIATGE-BASIC-1' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric goal. turning cost-minimizing relaxed plans OFF.

ff: search configuration is Enforced Hill-Climbing, then A*epsilon with weight 5.
Metric is ((1.00*[RF0](NUM-VISITED)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

Cueing down from goal distance:    3 into depth [1]
                                   2           [1]
                                   1           [1]
                                   0
ff: found legal plan as follows
step    0: CHOOSE-START BCN H-BCN
        1: TRAVEL BCN PAR H-PAR
        2: TRAVEL PAR ROM H-ROM
plan cost: 3.000000

time spent:    0.00 seconds instantiating 16 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 18 facts and 16 actions
               0.00 seconds creating final representation with 18 relevant facts, 1 relevant fluents
               0.04 seconds computing LNF
               0.00 seconds building connectivity graph
               0.00 seconds searching, evaluating 4 states, to a max depth of 1
               0.04 seconds total time

===== problem2 (graf en cadena, min 4) =====

ff: parsing domain file
domain 'VIATGES-BASIC' defined
... done.
ff: parsing problem file
problem 'VIATGE-BASIC-2' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric goal. turning cost-minimizing relaxed plans OFF.

ff: search configuration is Enforced Hill-Climbing, then A*epsilon with weight 5.
Metric is ((1.00*[RF0](NUM-VISITED)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

Cueing down from goal distance:    4 into depth [1]
                                   3           [1]
                                   2           [1]
                                   1           [1]
                                   0
ff: found legal plan as follows
step    0: CHOOSE-START MAD H-MAD
        1: TRAVEL MAD BCN H-BCN
        2: TRAVEL BCN PAR H-PAR
        3: TRAVEL PAR BER H-BER
plan cost: 4.000000

time spent:    0.00 seconds instantiating 18 easy, 0 hard action templates
               0.00 seconds reachability analysis, yielding 26 facts and 18 actions
               0.00 seconds creating final representation with 26 relevant facts, 1 relevant fluents
```

```
0.00 seconds computing LNF
0.00 seconds building connectivity graph
0.00 seconds searching, evaluating 5 states, to a max depth of 1
0.00 seconds total time
```

B.2. Traces Extensió 1

```
#####
# TRACES EXTENSIO 1 (dies min/max + dies recorregut) metric-ff v2.1
#####

===== problem1 (3 ciutats, dies 2-4, trip>=7) =====

ff: parsing domain file
domain 'VIATGES-EXT1' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT1-1' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is Enforced Hill-Climbing, then A*epsilon with weight 5.
Metric is ((1.00*[RF0](TOTAL-DAYS)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

Cueing down from goal distance:   4 into depth [1]
                                   3         [1]
                                   2         [1]
                                   1         [1]
                                   0
ff: found legal plan as follows
step   0: CHOOSE-START BCN H-BCN
        1: TRAVEL BCN PAR H-PAR
        2: TRAVEL PAR ROM H-ROM
        3: STAY-EXTRA-DAY ROM
plan cost: 7.000000

time spent:   0.00 seconds instantiating 12 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 14 facts and 12 actions
              0.00 seconds creating final representation with 14 relevant facts, 8 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 5 states, to a max depth of 1
              0.00 seconds total time

===== problem2 (4 ciutats cadena, dies 1-3, trip>=4) =====

ff: parsing domain file
domain 'VIATGES-EXT1' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT1-2' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is Enforced Hill-Climbing, then A*epsilon with weight 5.
Metric is ((1.00*[RF0](TOTAL-DAYS)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

Cueing down from goal distance:   4 into depth [1][2]
                                   3         [1]
                                   2         [1]
                                   1         [1]
                                   0
ff: found legal plan as follows
step   0: CHOOSE-START MAD H-MAD
        1: TRAVEL MAD BCN H-BCN
        2: TRAVEL BCN PAR H-PAR
        3: STAY-EXTRA-DAY PAR
        4: TRAVEL PAR BER H-BER
plan cost: 5.000000

time spent:   0.00 seconds instantiating 24 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 26 facts and 24 actions
              0.00 seconds creating final representation with 26 relevant facts, 14 relevant fluents
              0.00 seconds computing LNF
```

```
0.00 seconds building connectivity graph
0.00 seconds searching, evaluating 12 states, to a max depth of 2
0.00 seconds total time
```

B.3. Traces Extensió 2

```
#####
# TRACES EXTENSIO 2 (interès) (metric-ff v2.1, cerca: -s 3 -w 0 = cost uniforme/optim)
#####

===== problem1 (5 ciutats, minimitzar interès, min3) =====

ff: parsing domain file
domain 'VIATGES-EXT2' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT2-1' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((1.00*[RF0](TOTAL-INTEREST)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   3
                             2
                             1
                             0

ff: found legal plan as follows
step   0: CHOOSE-START PAR H-PAR
        1: TRAVEL PAR LON H-LON
        2: TRAVEL LON BCN H-BCN
plan cost: 4.000000

time spent:   0.01 seconds instantiating 30 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 22 facts and 30 actions
              0.00 seconds creating final representation with 22 relevant facts, 13 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 56 states, to a max depth of 0
              0.01 seconds total time

===== problem2 (graf parcial, min4) =====

ff: parsing domain file
domain 'VIATGES-EXT2' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT2-2' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((1.00*[RF0](TOTAL-INTEREST)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   4
                             3
                             2
                             1
                             0

ff: found legal plan as follows
step   0: CHOOSE-START AMS H-AMS
        1: TRAVEL AMS BER H-BER
        2: TRAVEL BER PAR H-PAR
        3: TRAVEL PAR BCN H-BCN
plan cost: 6.000000

time spent:   0.00 seconds instantiating 24 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 26 facts and 24 actions
              0.00 seconds creating final representation with 26 relevant facts, 15 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 41 states, to a max depth of 0
              0.00 seconds total time
```


B.4. Traces Extensió 3

```
#####
# TRACES EXTENSIO 3 (preus) (metric-ff v2.1, cerca: -s 3 -w 0 = cost uniforme/optim)
#####

===== problem1 (minimitzar preu, rang 100..500) =====

ff: parsing domain file
domain 'VIATGES-EXT3' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT3-1' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((1.00*[RF0](TOTAL-PRICE)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   3
                             2
                             0

ff: found legal plan as follows
step   0: CHOOSE-START ROM H-ROM
        1: TRAVEL F-RP ROM PAR H-PAR2
        2: TRAVEL F-PB2 PAR BCN H-BCN
plan cost: 310.000000

time spent:   0.00 seconds instantiating 28 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 33 facts and 28 actions
              0.00 seconds creating final representation with 33 relevant facts, 12 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 86 states, to a max depth of 0
              0.00 seconds total time

===== problem2 (preu minim restrictiu 380..600) =====

ff: parsing domain file
domain 'VIATGES-EXT3' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT3-2' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((1.00*[RF0](TOTAL-PRICE)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   5
                             2
                             1
                             0

ff: found legal plan as follows
step   0: CHOOSE-START LON H-LON
        1: TRAVEL F-LB LON BCN H-BCN
        2: TRAVEL F-BP2 BCN PAR H-PAR2
plan cost: 380.000000

time spent:   0.00 seconds instantiating 28 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 33 facts and 28 actions
              0.00 seconds creating final representation with 33 relevant facts, 12 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 108 states, to a max depth of 0
              0.00 seconds total time
```

B.5. Traces Extensió 4

```
#####
# TRACES EXTENSIO 4 (interès + preu ponderats) (metric-ff v2.1, cerca: -s 3 -w 0 = cost uniforme/optim)
#####

===== problem1 (ponderacio orientada al PREU) =====

ff: parsing domain file
domain 'VIATGES-EXT4' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT4-1' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((1.00*[RF0](TOTAL-INTEREST)1.00*[RF1](TOTAL-PRICE)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   3
                             2
                             1
                             0

ff: found legal plan as follows
step    0: CHOOSE-START AMS H-AMS
        1: TRAVEL F-AB AMS BCN H-BCN
        2: TRAVEL F-BL BCN LON H-LON
plan cost: 229.000000

time spent:   0.00 seconds instantiating 28 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 40 facts and 28 actions
              0.00 seconds creating final representation with 40 relevant facts, 15 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.00 seconds searching, evaluating 40 states, to a max depth of 0
              0.00 seconds total time

===== problem2 (ponderacio orientada a l'INTERES) =====

ff: parsing domain file
domain 'VIATGES-EXT4' defined
... done.
ff: parsing problem file
problem 'VIATGE-EXT4-2' defined
... done.

translating negated cond for predicate VISITED
translating negated cond for predicate STARTED
warning: numeric precondition. turning cost-minimizing relaxed plans OFF.

ff: search configuration is weighted A* with weight 0.
Metric is ((100.00*[RF0](TOTAL-INTEREST)1.00*[RF1](TOTAL-PRICE)) - () + 0.00)
COST MINIMIZATION DONE (WITHOUT cost-minimizing relaxed plans).

advancing to goal distance:   3
                             2
                             1
                             0

ff: found legal plan as follows
step    0: CHOOSE-START AMS H-AMS
        1: TRAVEL F-AP AMS PAR H-PAR
        2: TRAVEL F-PR PAR ROM H-ROM
plan cost: 1040.000000

time spent:   0.00 seconds instantiating 28 easy, 0 hard action templates
              0.00 seconds reachability analysis, yielding 40 facts and 28 actions
              0.00 seconds creating final representation with 40 relevant facts, 15 relevant fluents
              0.00 seconds computing LNF
              0.00 seconds building connectivity graph
              0.16 seconds searching, evaluating 72 states, to a max depth of 0
              0.16 seconds total time
```